Milestone Report

Self-Improving Agent Harnesses via Trace-Derived RL Environments for Custom-Kernel Engineering

Chandra Suda Aaditya Nalawade

CS 224R: Deep Reinforcement Learning, Stanford | May 23, 2026

1. Experiments so far

Our overarching objective is an ML-research agent that mines its own logs, diagnoses defects, synthesizes verifiable RL environments for those defects, and trains a lightweight LoRA policy that improves future harness behavior. For the milestone we narrowed the verifier domain to GPU kernel engineering (§2) and built kernel-synth (https://github.com/chandrasuda/kernel-synth), a trace-derived environment factory. The pipeline shallow-clones a GitHub repo, AST-walks every .py to enumerate nn.Module subclasses, and uses either our custom tool-use harness (list_files / read_file / mark_module / finish) or a deterministic AST heuristic — scoring forward ops, custom inits/buffers, name signals (mamba/ssm/rotary/hyena/…), and imports of triton / flash_attn / mamba_ssm / einops — to pick a diverse, non-obvious set per repo. A converter then emits one self-contained RL env per chosen module: reference.py (class + filtered imports + transitively-lifted helper defs), inputs.py (auto-inferred from ___init___/forward signatures), solution.py, harness.py (eager-vs-solution timing + torch.allclose), and env.json; a small FastAPI viewer browses the catalog. On top of this we wired a Harbor-style ATIF v1.7 rollout layer that uses torch.compile as the reward ceiling and exposes Triton-only agent tools (read_file / write_file / run_benchmark / finish) inside a workspace sandbox; baseline and torch.compile rollouts produce trajectories that round-trip through the schema validator.

1.1 Required experiment

We ran the pipeline end-to-end in deterministic-heuristic mode on five PyTorch repos chosen for mechanism diversity, wall-clock ≈ 11 s:

repo	py files	kept	avg novelty	top picks
state-spaces/mamba	55	8	0.83	Mamba2, Mamba3, MHA
lucidrains/vit-pytorch	76	8	0.92	DSSA, NaViT, LookViT
lucidrains/x-transformers	27	8	0.96	CoPE, DataDependentAlibi, HyperConnection
openai/whisper	14	3	0.21	(down-weighted: vanilla transformer)
lucidrains/audiolm-pytorch	14	8	0.75	Attend, SoundStream, HubertWithKmeans
total	186	35	0.81	∼60k LOC scanned

What worked. The heuristic correctly down-weighted openai/whisper (vanilla encoder/decoder ⇒ avg 0.21) and saturated near 1.0 on novel mechanisms (CoPE, DataDependentAlibi, Mamba2/3, DSSA) — the right inductive bias for our reward: we don't want envs on modules where a fused kernel already exists, since the speedup ceiling collapses. What didn't. The first converter naïvely copied the source file's full import block, breaking many envs via loguru/einx/etc. Fix: walk the class body for referenced names, keep only imports whose bound name is used, and transitively lift module-level helper defs (e.g. exists, default) until a fixed point. CoPE's reference.py now imports cleanly and exists() is lifted automatically — trace-derived envs are only useful if the reference module actually runs.

2. Changes to hypothesis / objective

The closed-loop hypothesis is unchanged: agents improve faster when their RL training environments are mined from real trace/code distributions than when they are hand-designed. What narrowed is the verifier — "general ML-research harness defects" → "GPU kernel engineering" — because kernels admit a cheap, fully-automatable, dense reward (numerical equivalence + wall-clock speedup vs torch.compile), which is exactly what makes long-horizon RL tractable; general ML-research decisions are subjective, slow to verify, and contaminated by API/cost noise, so they are better attacked once the loop is proven on the kernel sub-domain. Preserved: trace-derived envs, GRPO+LoRA training, evaluation vs base / SFT / prompt-only baselines, ablations over env-derivation vs synthetic. New: a concrete environment factory (kernel-synth), an external expert baseline (torch.compile) defining the reward ceiling, and Triton as a much smaller action space than the full agentic one — substantially easier to RL.

3. Steps to completion

(1) Harden the rollout layer: stress-test the per-env benchmark.py (eager + torch.compile + solution under torch.allclose), tighten the tool-use loop for the agent mode, and improve the auto-inferred inputs.py so more envs run end-to-end out of the box (the current set has hand-tuned shapes for only a few modules). (2) Generate trajectories at scale: run Claude-Sonnet and open-source coding LLMs over the 35 envs plus a held-out 5 repos to produce $N \approx 200$ ATIF trajectories with rewards in $\{-0.1\} \cup [-0.2, 1.5]$. (3) GRPO+LoRA training on Qwen3-1.7B or SmolLM3-3B with reward $r = \text{clip}((t_e - t_s)/(t_e - t_c), -0.2, 1.5)$ (t_e, t_s, t_c = eager/solution/compile latency) gated by correctness; evaluate on held-out repos vs base, prompt-only self-reflection, and SFT. (4) Ablations: (a) heuristic vs agent env-selection, (b) trace-derived envs vs a generic Triton-toy-task curriculum (KernelBench-mini style),

(c) torch.compile ceiling on/off. (5) Final eval: best-of-N solve rate, mean reward, fraction of held-out modules reaching $r \geq 0.5$, and the t_s/t_c distribution.